3.5. Policy Gradient and Related Methods . . . . . p. 408
3.5.1. Gradient Methods for Parametric Cost . . . . . Optimization . . . . . p. 409
3.5.2. Policy Gradient-Like Methods . . . . . p. 412
3.5.3. Scaling and Proximal Policy Optimization . . . . . Methods . . . . . p. 417
3.5.4. Random Direction Methods . . . . . p. 424
3.5.5. Random Search and Cross-Entropy Methods . . p. 426
3.5.6. Refining and Retraining Parametric Policies . . p. 428
3.6. Aggregation . . . . . p. 429
3.6.1. Aggregation with Representative States . . . . p. 430
3.6.2. Continuous Control Space Discretization . . . . p. 437
3.6.3. Continuous State Space - POMDP Discretization p. 438
3.6.4. General Aggregation . . . . . p. 440
3.6.5. Types of Aggregation and Error Bounds . . . . p. 443
3.6.6. Aggregation Using Features . . . . . p. 445
3.6.7. Biased Aggregation . . . . . p. 446
3.6.8. Asynchronous Distributed Aggregation . . . . . p. 449
3.7. Notes, Sources, and Exercises . . . . . p. 451

In this chapter, we will discuss the methods and objectives of off-line training through the use of parametric approximation architectures such as neural networks. We begin with a general discussion of parametric architectures and their training in Section 3.1. We then consider the training of neural networks in Section 3.2, and their use in the context of finite horizon approximate DP in Section 3.3. In Sections 3.4 and 3.5, we discuss the training of policies. Finally, in Section 3.6, we discuss aggregation methods.

3.1 PARAMETRIC APPROXIMATION ARCHITECTURES

For the success of approximation in value space, it is important to select a class of lookahead function approximations $\tilde{J}_k$ that is suitable for the problem at hand. In the preceding two chapters we discussed several methods for choosing $\tilde{J}_k$, based mostly on some form of rollout. We will now discuss how $\tilde{J}_k$ can be obtained by off-line training from a parametric class of functions, possibly involving a neural network, with the parameters "optimized" with the use of some algorithm.

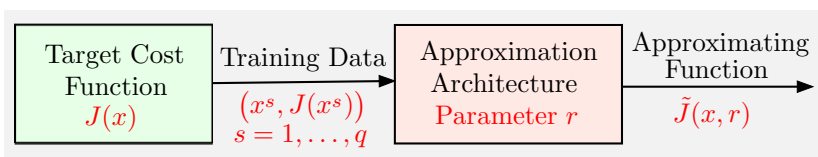

Figure 3.1.1 The general structure for parametric cost approximation. We approximate the target cost function $J(x)$ with a member from a parametric class $\tilde{J}(x, r)$ that depends on a parameter r. We use training data $\big(x^s, J(x^s)\big)$, $s = 1, \ldots, q$, and a form of optimization that aims to find a parameter $\hat{r}$ that "minimizes" the size of the errors $J(x^s) - \tilde{J}(x^s, \hat{r})$, $s = 1, \ldots, q$.

As we have noted in Chapter 1, the most popular structure for parametric cost function approximation involves a target function $J(x)$ that we want to approximate with a member of a parametric class of functions $\tilde{J}(x, r)$ that depends on a parameter r (see Fig. 3.1.1). In particular, we collect training data $\big(x^s, J(x^s)\big)$, $s = 1, \ldots, q$, which we use to determine a parameter $\hat{r}$ that leads to a good "fit" between the data $J(x^s)$ and the predictions $\tilde{J}(x^s, \hat{r})$ of the parametrized function. This is usually done through an optimization approach, aiming to minimize the size of the errors $J(x^s) - \tilde{J}(x^s, \hat{r})$, $s = 1, \ldots, q$.

Approximation of a target policy μ with a policy from a parametric class $\tilde{\mu}(x, r)$ is largely similar. Here, the training data may be obtained, for example, from rollout control calculations, thus enabling the construction of both value and policy networks that can be combined for use in a perpetual rollout scheme. An important difference, however, is that the

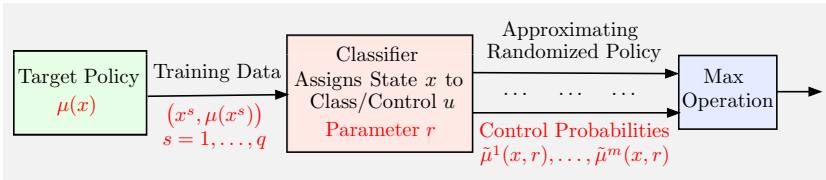

Figure 3.1.2 A general structure for parametric policy approximation for the case where the control space is finite, $U = \{u^1, \ldots, u^m\}$, and its relation to a classification scheme. It produces a randomized policy of the form (3.1), which is converted to a nonrandomized policy through the maximization operation (3.2).

approximate cost values $\tilde{J}(x,r)$ are real numbers, whereas the approximate policy values $\tilde{\mu}(x,r)$ are elements of a control space U. Thus if U consists of m dimensional vectors, $\tilde{\mu}(x,r)$ consists of m numerical components. In this case the parametric approximation problems for cost functions and for policies are fairly similar, and both involve continuous space approximations.

On the other hand, the case where the control space is finite $U = \{u^1, \ldots, u^m\}$ is quite different. In this case, for any x, $\tilde{\mu}(x,r)$ consists of one of the m possible controls $u^1, \ldots, u^m$. This connects policy space approximation with traditional classification schemes, whereby objects x are classified as belonging to one of the categories $u^1, \ldots, u^m$. In particular, $\tilde{\mu}(x,r)$ *provides the category of* x*, and* $\tilde{\mu}(\cdot,r)$ *can be viewed as a classifier*. Some of the most prominent classification schemes actually produce randomized outcomes, i.e., x is associated with a probability distribution

$$\{\tilde{\mu}^1(x,r), \ldots, \tilde{\mu}^m(x,r)\}, \tag{3.1}$$

which is a randomized policy in our policy approximation context; see Fig. 3.1.2. This is often done for algorithmic convenience, since many optimization methods, including least squares regression, require that the optimization variables are continuous. Then, the randomized policy (3.1) can be converted to a nonrandomized policy using a maximization operation: associate x with the control of maximum probability (cf. Fig. 3.1.2),

$$\tilde{\mu}(x,r) \in \arg\max_{i=1,\ldots,m} \tilde{\mu}^i(x,r). \tag{3.2}$$

We will discuss the use of classification methods for approximation in policy space in Section 3.4, following our discussion of parametric approximation in value space.

3.1.1 Cost Function Approximation

For the remainder of this section, as well as Sections 3.2 and 3.3, we will focus on approximation in value space schemes, where the approximate cost

functions are selected from a parametric class of functions $\tilde{J}_k(x_k, r_k)$ that for each k, depend on the current state x_k and a vector $r_k = (r_{1,k}, \ldots, r_{m_k,k})$ of m_k "tunable" scalar parameters. By adjusting these parameters, the "shape" of $\tilde{J}_k$ can be adjusted to closely approximate a target function, typically the true optimal cost-to-go function J_k^* or the cost-to-go function Jk, π of a given policy π. The class of functions $\tilde{J}_k(x_k, r_k)$ is called an *approximation architecture*, and the process of choosing the parameter vectors r_k is known as *training* or *tuning* the architecture. We will focus initially on approximation of cost functions, hence the use of the $\tilde{J}_k$ notation. In Section 3.4 we will consider the other major use of parametric approximation architectures, of the form $\tilde{\mu}_k(x_k, r_k)$, where the target function is a control function μ_k that is part of some policy.

The simplest training approach for parametric architectures is to do some form of semi-exhaustive or semi-random search in the space of parameter vectors and adopt the parameters that result in best performance of the associated one-step lookahead controller (according to some criterion). Such methods are mainly practical when the number of parameters is relatively small.

Random search and Bayesian optimization methods have also been used to tune *hyperparameters* of an approximation architecture. Examples include determining the number of layers in a neural network or selecting the number of clusters when partitioning discrete spaces. For further details on these methods, we refer to the research literature.

More systematic training methods are based on numerical optimization, such as a least squares fit that seeks to match the architecture's cost approximation to a "training set," i.e., a large number of state and cost value pairs obtained through some form of sampling. In Sections 3.1-3.3 we will focus primarily on this approach.

3.1.2 Feature-Based Architectures

There is a large variety of approximation architectures, including those based on polynomials, wavelets, radial basis functions, discretization/interpolation schemes, neural networks, and others. A particularly interesting type of cost approximation involves *feature extraction*, a process that maps the state x_k into some vector $\phi_k(x_k)$, called the *feature vector* associated with x_k at time k. The vector $\phi_k(x_k)$ consists of scalar components

$$\phi_{1,k}(x_k), \ldots, \phi_{m_k,k}(x_k),$$

called *features*. A feature-based cost approximation has the form

$$\tilde{J}_k(x_k, r_k) = \hat{J}_k\big(\phi_k(x_k), r_k\big),$$

where r_k is a parameter vector and $\hat{J}_k$ is some function. Thus, the cost approximation depends on the state x_k through its feature vector $\phi_k(x_k)$.

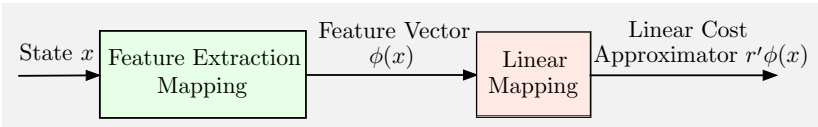

Figure 3.1.3 The structure of a linear feature-based architecture. At time k, we use a feature extraction mapping to generate an input $\phi_k(x_k)$ to a linear mapping defined by a parameter vector r_k.

Note that we are allowing for different features $\phi_k(x_k)$ and different parameter vectors r_k for each stage k. This is necessary for nonstationary problems (e.g., if the state space changes over time), and also to capture the effect of proximity to the end of the horizon. In contrast, for stationary problems with long or infinite horizons where the state space remains constant over k, it is common to use the same features and parameter vectors across all stages. The following discussion can be easily extended to accommodate infinite horizon methods, which we will explore in detail later.

Features are often designed manually, leveraging human intelligence, insights, and experience to capture the most significant characteristics of the current state. There are also systematic ways to construct features, including the use of data and neural networks, which we will discuss shortly. In this section, we provide a brief and selective presentation of architectures.

The rationale behind using features is that optimal cost-to-go functions J_k^* can be complex, nonlinear mappings. By decomposing their complexity into smaller, more manageable components, it becomes feasible to approximate these functions effectively. In particular, if the features encode much of the nonlinearity of J_k^*, we may be able to use a relatively simple architecture $\hat{J}_k$ to approximate J_k^*. For example, with a well-chosen feature vector $\phi_k(x_k)$, a good approximation to the cost-to-go is often provided by *linearly* weighting the features, i.e.,

$$\tilde{J}_k(x_k, r_k) = \hat{J}_k\big(\phi_k(x_k), r_k\big) = \sum_{\ell=1}^{m_k} r_{\ell,k}\phi_{\ell,k}(x_k) = r_k'\phi_k(x_k), \qquad (3.3)$$

where $r_{\ell,k}$ and $\phi_{\ell,k}(x_k)$ are the ℓth components of r_k and $\phi_k(x_k)$, respectively, and $r_k'\phi_k(x_k)$ denotes the inner product of r_k and $\phi_k(x_k)$, viewed as column vectors of $\Re^{m_k}$ (a prime denotes transposition, so r_k' is a row vector); see Fig. 3.1.3.

This is called a *linear feature-based architecture*, and the scalar parameters $r_{\ell,k}$ are also called *weights*. Among other advantages, these architectures admit simpler training algorithms than their nonlinear counterparts; see the neuro-dynamic programming (NDP) book [BeT96]. Mathematically, the approximating function $\tilde{J}_k(x_k, r_k)$ can be viewed as a member of the subspace spanned by the features $\phi_{\ell,k}(x_k)$, $\ell = 1, \ldots, m_k$, which

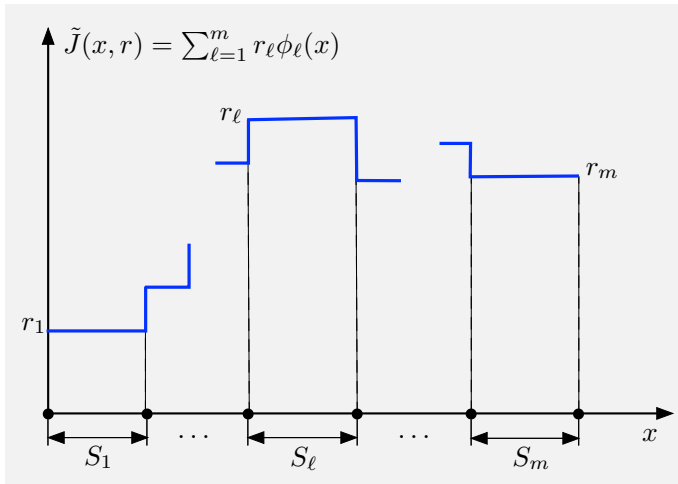

Figure 3.1.4 Illustration of a piecewise constant architecture. The state space is partitioned into subsets $S_1, \ldots, S_m$, with each subset S_ℓ defining the feature

$$\phi_\ell(x) = \begin{cases} 1 & \text{if } x \in S_\ell, \\ 0 & \text{if } x \notin S_\ell, \end{cases} \qquad \ell = 1, \ldots, m,$$

with its own weight r_ℓ.

for this reason are also referred to as *basis functions*. We provide a few examples, where for simplicity we drop the index k.

Example 3.1.1 (Piecewise Constant Approximation)

Suppose that the state space is partitioned into subsets $S_1, \ldots, S_m$, so that every state belongs to one and only one subset. Let the ℓth feature be defined by membership to the set S_ℓ, i.e.,

$$\phi_\ell(x) = \begin{cases} 1 & \text{if } x \in S_\ell, \\ 0 & \text{if } x \notin S_\ell, \end{cases} \qquad \ell = 1, \ldots, m.$$

Consider the architecture

$$\tilde{J}(x,r) = \sum_{\ell=1}^{m} r_\ell \phi_\ell(x),$$

where r is the vector consists of the m scalar parameters $r_1, \ldots, r_m$. It can be seen that $\tilde{J}(x,r)$ is the piecewise constant function that has value r_ℓ for all states within the set S_ℓ; see Fig. 3.1.4.

The piecewise constant approximation is an example of a linear feature-based architecture that involves exclusively *local features*. These are

features that take a nonzero value only for a relatively small subset of states. Thus a change of a single weight causes a change of the value of $\tilde{J}(x,r)$ for relatively few states x. At the opposite end we have linear feature-based architectures that involve *global features*. These are features that take nonzero values for a large number of states. The following is a common example.

Example 3.1.2 (Polynomial Approximation)

An important case of linear architecture is one that uses polynomial basis functions. Suppose that the state consists of n components $x^1, \ldots, x^n$, each taking values within some range of integers. For example, in a queueing system, x^i may represent the number of customers in the ith queue, where $i = 1, \ldots, n$. Suppose that we want to use an approximating function that is quadratic in the components x^i. Then we can define a total of $1 + n + n^2$ basis functions that depend on the state $x = (x^1, \ldots, x^n)$ via

$$\phi_0(x) = 1, \qquad \phi_i(x) = x^i, \qquad \phi_{ij}(x) = x^i x^j, \quad i,j = 1, \ldots, n.$$

A linear approximation architecture that uses these functions is given by

$$\tilde{J}(x,r) = r_0 + \sum_{i=1}^{n} r_i x^i + \sum_{i=1}^{n} \sum_{j=1}^{n} r_{ij} x^i x^j,$$

where the parameter vector r has components r_0, r_i, and r_{ij}, with $i,j = 1, \ldots, n$. Indeed, any kind of approximating function that is polynomial in the components $x^1, \ldots, x^n$ can be constructed similarly.

A more general polynomial approximation may be based on some other known features of the state. For example, we may start with a feature vector

$$\phi(x) = \big(\phi_1(x), \ldots, \phi_m(x)\big)',$$

and transform it with a quadratic polynomial mapping. In this way we obtain approximating functions of the form

$$\tilde{J}(x,r) = r_0 + \sum_{i=1}^{m} r_i \phi_i(x) + \sum_{i=1}^{m} \sum_{j=1}^{m} r_{ij} \phi_i(x) \phi_j(x),$$

where the parameter r has components r_0, r_i, and r_{ij}, with $i,j = 1, \ldots, m$. This can also be viewed as a linear architecture that uses the basis functions

$$w_0(x) = 1, \qquad w_i(x) = \phi_i(x), \qquad w_{ij}(x) = \phi_i(x)\phi_j(x), \quad i,j = 1, \ldots, m.$$

The preceding example architectures are generic in the sense that they can be applied to many different types of problems. Other architectures rely on problem-specific insight to construct features, which are then combined into a relatively simple architecture. We present two examples involving games.